

Lecture 12: Nonlinear least-squares

Jamie Haddock

Table of contents

1 Nonlinear Least Squares	1
1.1 Nonlinear least squares	1
1.2 Gauss-Newton method	1
1.3 Implementation	2
1.4 Nonlinear data fitting	3

1 Nonlinear Least Squares

Just as with linear systems, we can sometimes fail to find an exact solution and need to find a least-squares solution, rootfinding problems can sometime fail to have an exact solution and we must turn instead to an optimization problem called **nonlinear least-squares**.

1.1 Nonlinear least squares

Definition: Nonlinear least-squares problem

Given a function $\mathbf{f}(\mathbf{x})$ mapping from $\mathbb{R}^n$ to $\mathbb{R}^m$, the **nonlinear least-squares problem** is to find $\mathbf{x} \in \mathbb{R}^n$ such that $\|\mathbf{f}(\mathbf{x})\|_2$ is minimized.

This problem is equivalent to minimizing the square of the objective (since it is already positive) and so we may instead minimize the objective function $\phi(\mathbf{x}) = \mathbf{f}(\mathbf{x})^\top \mathbf{f}(\mathbf{x})$.

1.2 Gauss-Newton method

Just as with the rootfinding problem, we may substitute a linear model function in place of $\mathbf{f}$.

At estimate $\mathbf{x}_k$, we define the linear model

$$\mathbf{q}(\mathbf{x}) = \mathbf{f}(\mathbf{x}_k) + \mathbf{A}_k(\mathbf{x} - \mathbf{x}_k),$$

where $\mathbf{A}_k$ is the $m \times n$ Jacobian matrix or an approximation of it using the techniques from Lecture 11.

In the rootfinding problem, we sought a next iterate that satisfied $\mathbf{q}(\mathbf{x}_{k+1}) = \mathbf{0}$, but now we aim to define $\mathbf{x}_{k+1}$ to be the value that minimizes $\|\mathbf{q}(\mathbf{x})\|_2$. This defines the **Gauss-Newton method**.

Given $\mathbf{f}$ and a starting value $\mathbf{x}_1$, for each $k = 1, 2, 3, \dots$,

1. Compute $\mathbf{y}_k = \mathbf{f}(\mathbf{x}_k)$ and $\mathbf{A}_k$, the exact or approximate Jacobian matrix at $\mathbf{x}_k$.
2. Solve the linear least-squares problem $\operatorname{argmin} \|\mathbf{A}_k \mathbf{s}_k + \mathbf{y}_k\|_2$ for $\mathbf{s}_k$.
3. Let $\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{s}_k$.

1.3 Implementation

Our `newtonsys` and `levenberg` functions work as Gauss-Newton with exact Jacobian and approximate Jacobian, respectively, since we used the `\` operator, which solves the linear least-squares problem!

```
newtonsys
```

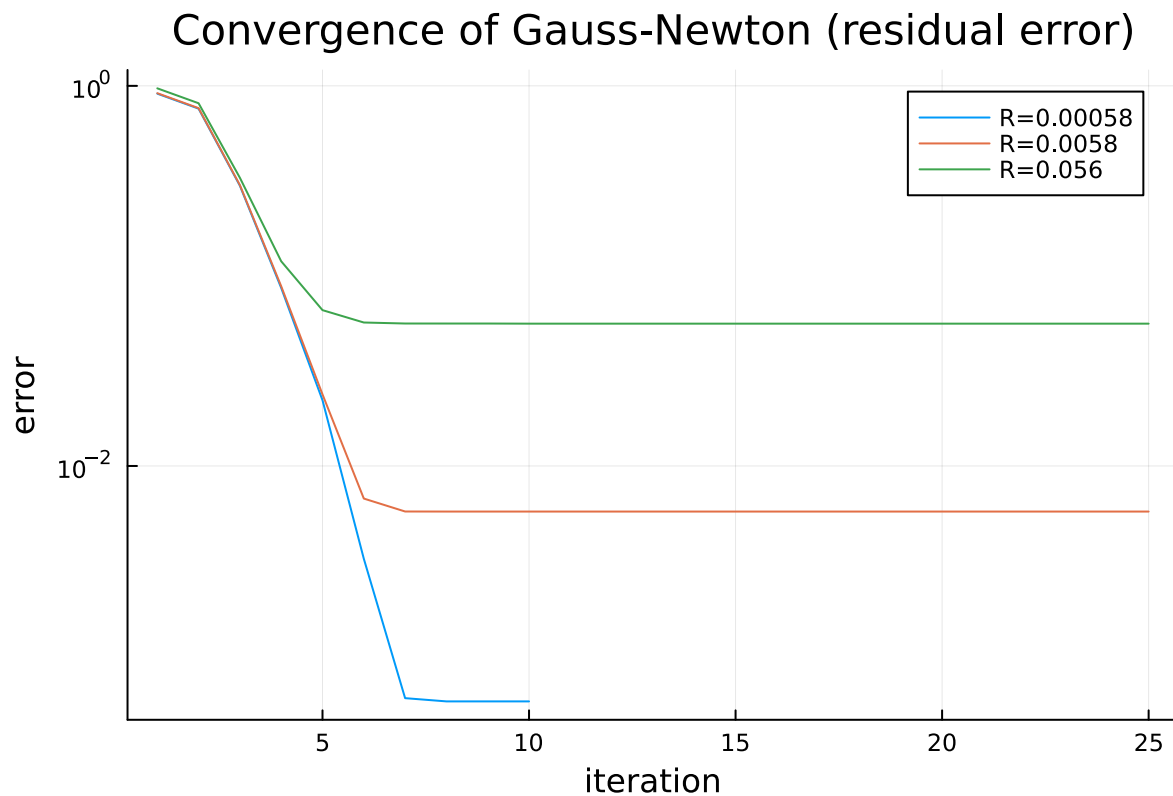
```
fdjac
```

```
levenberg
```

```
g(x) = [sin(x[1]+x[2]),cos(x[1]-x[2]),exp(x[1]-x[2])]
p = [1,1];
```

The function $\mathbf{g}(\mathbf{x}) - \mathbf{g}(\mathbf{p})$ has zero residual at $\mathbf{p}$. We make perturbations of this function to create nonzero residuals.

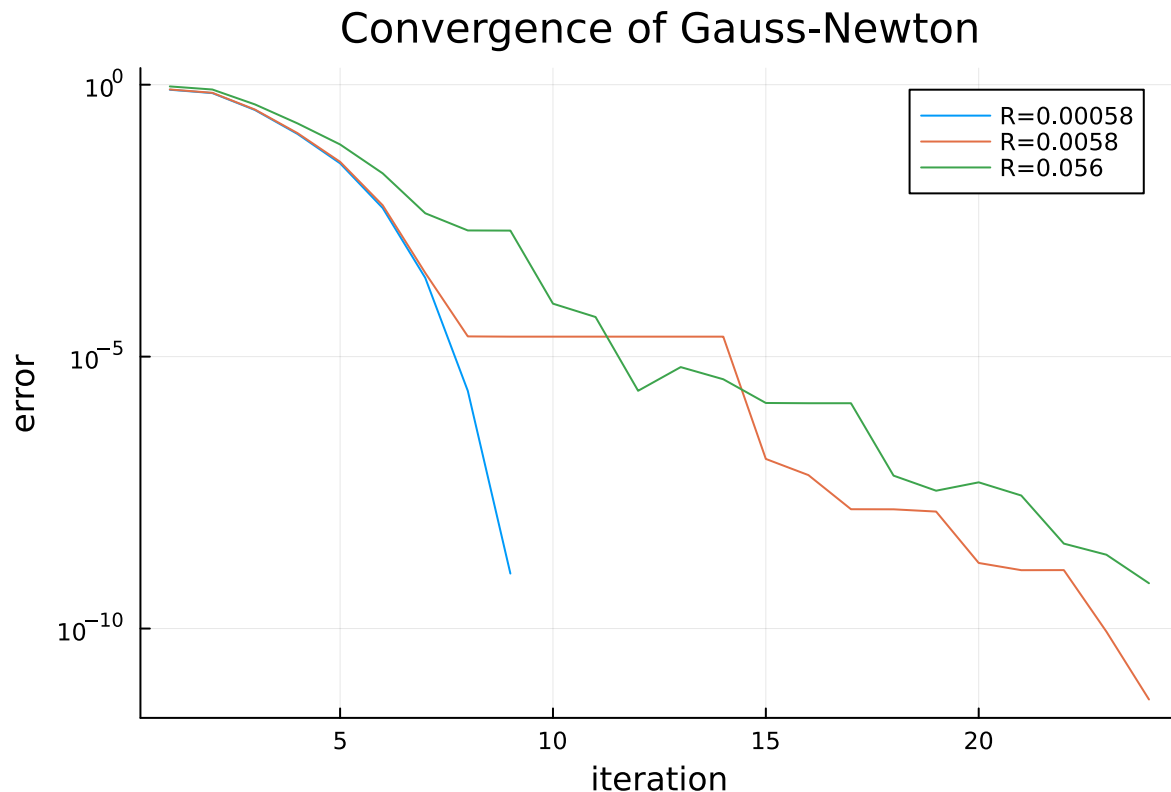
```
plt = plot(xlabel="iteration",yaxis=(:log10,"error"), title="Convergence of Gauss-Newton (residual error)
for R in [1e-3, 1e-2, 1e-1]
    # Define the perturbed function.
    f(x) = g(x) - g(p) + R*normalize([-1,1,-1])
    x = levenberg(f,[0,0])
    r = x[end]
    res_err = @. norm(f(x))
    normres = norm(f(r))
    plot!(res_err,label=@sprintf("R=%.2g",normres))
end
plt
```



```

plt = plot(xlabel="iteration",yaxis=(log10,"error"), title="Convergence of Gauss-Newton")
for R in [1e-3, 1e-2, 1e-1]
    # Define the perturbed function.
    f(x) = g(x) - g(p) + R*normalize([-1,1,-1])
    x = levenberg(f,[0,0])
    r = x[end]
    err = [norm(x-r) for x in x[1:end-1]]
    normres = norm(f(r))
    plot!(err,label=@sprintf("R=%.2g",normres))
end
plt

```



In the least perturbed case, the convergence is roughly quadratic, and the next least perturbed case exhibits initially quadratic and then linear convergence, while the most perturbed case exhibits nearly entirely linear convergence.

1.4 Nonlinear data fitting

Suppose that (t_i, y_i) for $i = 1, \dots, m$ are given data points. We wish to model this data by a function $g(t, \mathbf{x})$ that depends upon the unknown parameters $x_1, x_2, \dots, x_n$ in an arbitrary (nonlinear) way.

As in the linear case, a natural approach is to minimize the discrepancy between the data y_i and the model output $g(t_i, \mathbf{x})$ in a least-squares sense. We define the **misfit** function

$$\mathbf{f}(\mathbf{x}) = [g(t_i, \mathbf{x}) - y_i]_{i=1, \dots, m}$$

and seek to minimize $\|\mathbf{f}(\mathbf{c})\|_2^2$ over all coefficient vectors $\mathbf{c}$.

Note that the choice of form of the function g is up to the modeler and may be chosen given understanding of the data and underlying mechanisms, or simply so that g has enough algebraic complexity to fit to the

data well.

If the dependence on $\mathbf{c}$ is linear,

$$g(t, \mathbf{c}) = c_1 g_1(t) + c_2 g_2(t) + \cdots + c_m g_m(t),$$

then $\operatorname{argmin} \|\mathbf{f}(\mathbf{c})\|_2^2$ is a *linear* least-squares problem.

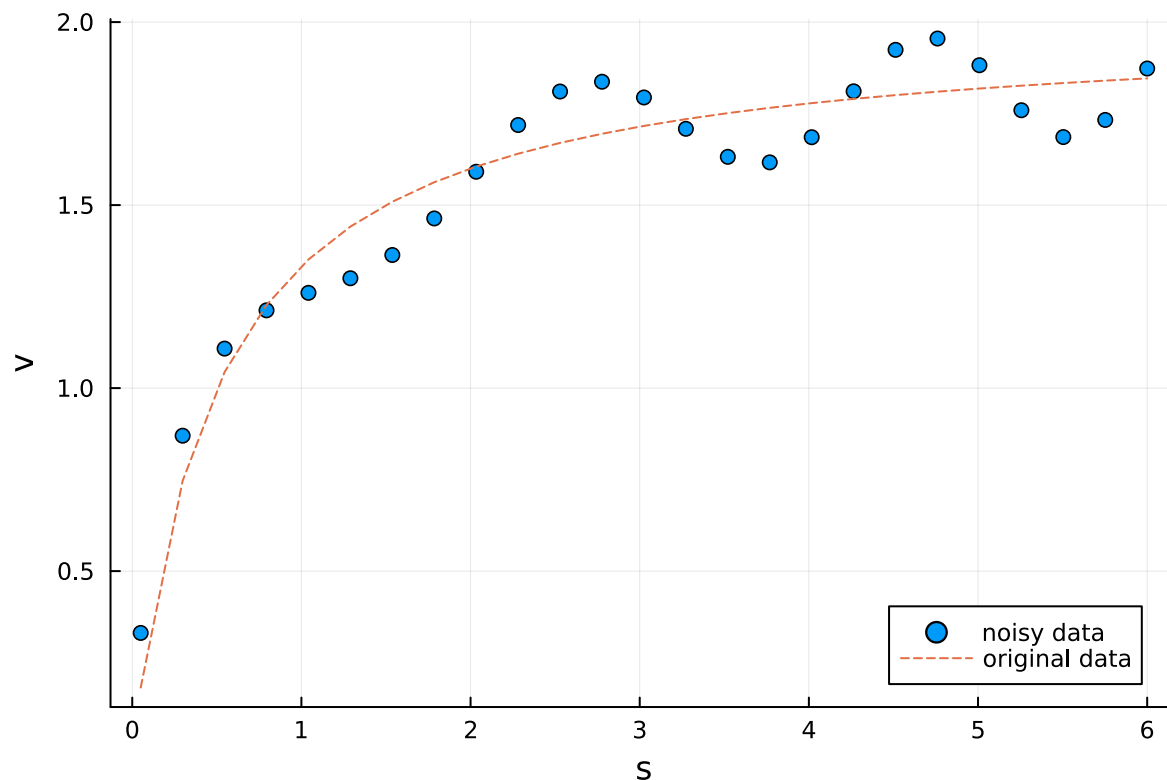
<https://youtu.be/3OKIZIGZZkg>

In this demo, we'll play with data representing the enzyme reaction rate $v(s)$ under the *Michaelis-Menten* kinetics with $V = 2$ and $K_m = 1/2$,

$$v(s) = \frac{Vs}{K_m + s}.$$

```
m = 25;
s = range(0.05, 6, length=m)
v_hat = @. 2*s/(0.5+s)           # exact data
v = @. v_hat + 0.15*cos(2*exp(s/16)*s); # smooth additive noise
```

```
scatter(s, v, label="noisy data", xlabel="s", ylabel="v", leg=:bottomright)
plot!(s, v_hat, l=:dash, label="original data")
```



We'll now imagine that we only have access to the noisy data and attempt to use nonlinear least-squares to learn the parameters in the model function $v(s)$. Here, v is our g function, s is our t variable, and the coefficients $\mathbf{c}$ are the values of V and K_m .

```
function misfit(x)
    V, Km = x      #rename components for clarity
    return @. V*s/(Km+s) - v
end
```

misfit (generic function with 1 method)

```
function misfitjac(x)
    V, Km = x      #rename components for clarity
    J = zeros(m,2)
    J[:,1] = @. s/(Km+s)      # dw/dV
    J[:,2] = @. -V*s/(Km+s)^2 # dw/dKm
    return J
end
```

misfitjac (generic function with 1 method)

```
x = [1, 0.75]
x = newtonsys(misfit,misfitjac,x)

@show V, Km = x[end]    # final values
```

(V, Km) = x[end] = [1.96865259837822, 0.46930373074166293]

2-element Vector{Float64}:

1.96865259837822

0.46930373074166293

```
model = s -> V*s/(Km+s)
```

#9 (generic function with 1 method)

```
plot!(model,0,6,label="nonlinear fit")
```

